

```
# Calculating the area of a circle, v3

pi = 3.14159
r = input('What is the radius? ')      # the
question is optional
area = pi*(r**2)

if          area          >          100:
    # a simple condition.
    Remember the colon at the end of the statement
    print("That's one big circle! Its area is:",area) # notice
    that we use double quotes here, because we've got a single quote
    in the sentence — don't want to trip up the interpreter!
else:
    print("Bah. That's tiny. Just",area)
***code***
```

Row, if you enter a radius that's 6 or more, you'll be complimented on your choice. As you can see, Python's "if.." statement is pretty simple. Every statement that you indent after the "if" will be executed if the condition you specify is true. The same goes for "else" — every indented statement after it will be executed if the condition is false.

When you're done mucking about with conditional statements, you can take a ride on one of Python's loops.

Round And Round

Writing loops in Python is as easy as conditional statements. One function that's going to come to your aid here is "range", which lets you specify a starting number, an end, and a step that's added to every number. For example, range(1,100,2) starts at 1, and goes up to 100 in steps of 2. Let's look at a simple loop:

```
***code***
# Counting to 50 in steps of 4
for i in range(0,50,4):          #again, remember
the colon
    print(i)
***code***
```

That's really all there is to it — pretty straightforward, yes? Python even has a "while" loop that's just as easy:

```
***code***
# Counting with a while loop
i=0          # you have to assign variables
first
while i<=50:
    print(i)
    i=i+4    # or i+=4, if you prefer
***code***
```

Unlike other languages, Python doesn't have a "do-while" loop, so if you want your block of code to run at least once, you'll have to figure out a smart way to do a while loop.

All right, enough with the trivial stuff.

Getting Functional

When you're programming in the real world, you're not going to write new programs every time you need to calculate the area of a circle, or want to count to 100 — you're going to write *functions*, which are like little bottles of code that you can keep handy, and use them when you need to. Here's how you define a function in Python:

```
***code***
# A function to calculate the area of a rect-
angle
def area_rect(length,breadth):      # accepting
~arguments~ from your main code
    a = length*breadth
    return a                        # sending
back the result of the calculation
```

Now, you can use this function anywhere in the main code

```
l = input('What is the length? ')
b = input('What is the breadth? ')
area = area_rect(l,b)
print('The area is', area)
***code***
```

Here's where Python's modular nature comes through. If you want to write a bunch of functions that you can use in any program that you write, you just need to write one module, and import it into the other modules you write. First, let's write a module that defines the area functions we need:

```
***code***
# module with functions for area
```

```
def area_rect(l,b):
    a=l*b
    return a
```

```
def area_circ(r):
    pi=3.14159
    a=pi*(r**2)
***code***
```

Save this as areas.py. Now, start a new module, in which we'll calculate the area of a circle:

```
***code***
# calculate the area of a circle
```

```
import areas          # gives this
module access to the functions in areas.py
```

```
radius = input('Enter a radius: ')
area = areas.area_circ(r)      # to access func-
tions, you need to type module.name.function
print(area)
***code***
```

Save this file — call it circle.py — in the same directory as areas.py. Now, when you run circles.py, Python will first compile areas.py into Python *bytecode* (you'll see a new file called areas.pyc), and then run the code in circles.py.

If you're going to use just a few functions of a module, however, it doesn't make sense to import the whole module. In the example above, we aren't using the area_rect function at all, so why import it? In this particular case, we have nothing to lose — but for more complex modules, you don't want a bunch of unnecessary code taking up your system memory. Thankfully, Python lets you import specific items from modules. Instead of the "import" statement, you just need to use "from ... import ...":

```
***code***
from areas import area_circ      # import just
this function
area = area_circ(5)              # you don't need
to use module.function() here
***code***
```

Welcome To The Jungle

Now that you've had a dose of the basics, you can move on to more advanced stuff, like object-oriented programming (OOP). If you haven't OOP-ed before, check out our *Fast Track to C++* for an introduction. You can also use Python in combination with a GUI framework — wxWidgets, Qt and the like — to build graphical applications that will run on Windows, Mac and Linux. If you want a change from PHP, you can even use Python to power your web site as well. If you use Blender for 3D graphics, you can have all sorts of fun with it using Python scripts. To cut a long story short, if you want to do it, chances are that you can do it with Python. ■

readersletters@thinkdigit.com